

“Food Donation and NGO Connect Platform”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree
of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

K. SATHWIKA

2520090210

A. SANJANA

2520090159

S. ANUSHKA

2520090168

Under the Esteemed Guidance of

Divya Priya Degala

Assistant Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Food Donation and NGO Connect Platform”** is a record of bonafide work of **K. SATHWIK**A – 2520090210, **A. SANJANA** – 2520090159, **S. ANUSHKA** – 2520090168 submitted in partial fulfillment of the requirements of the course DATABASE MANAGEMENT SYSTEMS, Section 10, Team 18, during the academic year 2026–2027, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

K. SATHWIKA – 2520090210

A. SANJANA – 2520090159

S. ANUSHKA – 2520090168

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled **“Food Donation and NGO Connect Platform”** is a bonafide work done and submitted by **K. SATHWIK**A – 2520090210, **A. SANJANA** – 2520090159, **S. ANUSHKA** – 2520090168 in partial fulfillment of the requirements of the course DATABASE MANAGEMENT SYSTEMS, Section 10, Team 18, at KLH University during the academic year 2026–2027.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance of many people. We wish to express our sincere thanks to everyone who assisted us, in one way or another, in completing this project.

Our greatest appreciation goes to our Course Coordinator, [Course Coordinator Name], and our guide, Divya Priya Degala, Assistant Professor, Department of Computer Science and Information Technology, for their tremendous support, encouragement and guidance throughout this project.

We express our gratitude to Dr. Kaja Shareef, Head of the Department of Computer Science and Information Technology, for providing us with the facilities and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff who assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

K. SATHWIKA – 2520090210

A. SANJANA – 2520090159

S. ANUSHKA – 2520090168

Abstract

The Food Donation and NGO Connect Platform is a database-driven web application that connects surplus-food donors — restaurants, caterers, event organisers and households — with nearby NGOs and volunteers before the food spoils. In most cities, edible surplus food is discarded every day while NGOs struggle to learn about it in time; coordination by phone calls and message groups is slow, does not scale, and leaves no structured record of what was offered, to whom, and what happened to it.

The proposed system stores donors, NGOs, volunteers, food listings and pickups in a normalised (3NF) MySQL schema of twelve tables, and places the matching intelligence inside the database itself. A stored function computes the Haversine distance between localities; a stored procedure applies four hard filters (food type accepted, distance within the NGO's service radius, quantity within the NGO's batch range, and food reachable before it expires) and then ranks the surviving NGOs with a weighted priority score (50% proximity, 30% quantity, 20% urgency). Every candidate considered is written to an audit table, so each match can be explained afterwards. Database triggers validate new listings, notify every eligible NGO automatically, and keep listing, pickup and volunteer states consistent as a donation moves from AVAILABLE to MATCHED, PICKED_UP and DELIVERED, with a complete status-history trail.

The application layer is built with Java 17 and Spring Boot 3.3 (Spring Web, Spring Data JPA, Spring Security with JWT authentication, and Flyway for versioned schema migration), and the user interface is a single-page web front end served by the same application on one port. It offers a public live-matching demonstration and a logged-in donor dashboard for posting donations, confirming pickups, tracking delivery and viewing history. The result is a verified donor-NGO network with real-time donation tracking and a transparent, reproducible matching process that reduces the time food spends unclaimed.

Index

S.No.	Chapters	Topics	Page No.
	Acknowledgement		4
	Abstract		5
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience 1.7 Literature Survey	7
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	9
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 Version Control 3.5 Authentication and Security Libraries	10
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Deployment Architecture	12
5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 Normalization 5.4 Core DBMS Concepts Used 5.5 Design Decisions and Assumptions	13
6	Implementation	6.1 Module-wise Implementation 6.2 Matching Engine (Stored Procedures) 6.3 Triggers 6.4 Front-end Logic 6.5 API Integration 6.6 Authentication and Authorization	16
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	20
8	Testing	8.1 API Testing Approach 8.2 Database-level Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results 8.5 System Performance Analysis	22
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	24
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	25
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	27
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	28
13	References	Documentation, tools, and resources used	29
14	Appendices	A: Sample Code Snippets B: Installation / Setup Instructions C: Seed Data and Accounts D: API Reference Summary E: Screenshots	30

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Food wastage and hunger exist side by side in the same cities. Restaurants, catering services, wedding halls and ordinary households routinely end up with cooked or packaged food that is still safe to eat but will not be consumed before it spoils. At the same time, shelters, community kitchens and charitable trusts regularly run short of food. What is missing is not the willingness of either side but a fast, reliable way for them to find each other.

Today that coordination is mostly informal: phone calls, WhatsApp groups and word of mouth. It works when a volunteer happens to be free and a message happens to be noticed, and it fails exactly when food is time-critical. There is no shared record of what was offered, which NGO was able to accept it, who collected it, or whether it ever reached anyone.

The Food Donation and NGO Connect Platform replaces this with a single relational database as the source of truth. Donors, NGOs, volunteers, food listings and pickups are stored in one schema, and the rules that decide which NGO should receive which donation are implemented inside the database as stored procedures and triggers, next to the data they operate on.

1.2 Problem Statement

Given a city with many food donors, a smaller number of NGOs that each accept only certain kinds of food, within a limited service area and within a limited batch size, and a small pool of volunteers who can transport food, the system must answer three questions quickly and consistently: (1) which NGOs can actually accept a given donation, given its food type, quantity, location and remaining shelf life; (2) which of those NGOs is the best choice; and (3) once a choice is confirmed, who will collect the food and what is the status of that pickup at any moment. Answering these questions manually does not scale, is error-prone, and leaves no audit trail.

1.3 Scope of the Project

The project covers the full lifecycle of a donation: registering and storing donors, NGOs, volunteers, localities and food types; posting a food listing; automatically validating it and notifying eligible NGOs; ranking eligible NGOs with a transparent scoring rule; confirming a pickup and assigning the nearest available volunteer; tracking the pickup through PICKED_UP and DELIVERED; and recording a complete audit trail of every match candidate and every status change. User accounts are secured with JWT-based authentication.

Out of scope for this submission: real-time GPS tracking and street-level geocoding (locations are locality centroids), payment or tax-receipt handling, delivery of notifications through e-mail/SMS/push (notifications are stored in the database), role-based access for donor/NGO/volunteer accounts, and native mobile applications.

1.4 Objectives

- Let NGOs discover surplus food donations from restaurants, households and event organisers as soon as they are posted.

- Match each donation to eligible NGOs using relational queries on location, food type, quantity and expiry window.
- Rank eligible NGOs with a single, documented weighted priority score and classify each as MATCH, REVIEW or WEAK.
- Notify all eligible NGOs automatically through a database trigger when a valid listing is inserted.
- Assign the nearest available volunteer when a pickup is confirmed, and prevent the same listing from being picked up twice.
- Track every donation from listing to delivery and keep an audit trail of match decisions and status changes.
- Enforce data validity at the database level (constraints, triggers, state-transition checks) rather than relying only on application code.

1.5 Importance of the Application

Surplus food is perishable, so the value of a platform like this is measured in minutes. By filtering out NGOs that cannot physically reach the food before it expires and ranking the rest, the system removes the guesswork from “who should I call first?”. Because every candidate and every decision is logged, a coordinator can later explain why a particular NGO was chosen, which builds trust between donors and NGOs and gives the organisation data it can use to plan future collection.

1.6 Target Users / Audience

- **Donors** – restaurants, caterers, event organisers and households that post surplus food.
- **NGOs** – shelters, community kitchens and trusts that receive food and are notified about nearby donations.
- **Volunteers** – people who collect food from the donor and deliver it to the NGO; the nearest free volunteer is assigned automatically.
- **Coordinators / evaluators** – users of the live matching demo and the audit tables, who inspect how and why matches were produced.

1.7 Literature Survey

Food-redistribution coordination has passed through three broad stages:

- **Generation 1 – Manual coordination.** Phone calls and messaging groups link donors to NGOs. The approach breaks down whenever a volunteer is unavailable or a message gets buried, and nothing is recorded in a structured form.
- **Generation 2 – Basic listing applications.** Simple apps let donors post surplus food, but offer no structured matching, ranking or delivery-tracking logic; NGOs must browse and decide for themselves.
- **Current approach – Relational-DBMS platform.** A structured schema, SQL-driven matching and trigger-based automation connect donors to NGOs reliably and with full traceability. This project follows this approach.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Component	Requirement	Reason
Processor	Multi-core CPU (Intel i5/i7 or AMD Ryzen 5 or better)	Concurrent query processing and Maven builds
Memory	8 GB RAM minimum (16 GB recommended)	Spring Boot application, MySQL server and browser run together; simultaneous donor and NGO sessions
Storage	256 GB SSD	Fast database reads/writes; space for Maven repository, MySQL data and source
Display	Full HD (1920×1080)	Dashboard and interface testing

Scalability note. For a city-wide deployment the database can be hosted on a managed service such as AWS RDS or Azure SQL, which can scale storage and concurrent connections elastically during festival or other high-donation periods.

2.2 Software Requirements

Category	Required Software / Tool
Programming language	Java 17 or later (application layer)
Build tool	Apache Maven 3.9 or later
Database	MySQL 8 (RDBMS) – must allow the migration user to create triggers and functions (the root user can)
Query & admin tools	MySQL Workbench / MySQL command-line client
Backend framework	Spring Boot 3.3.4 (REST APIs)
Frontend	Single-page HTML / CSS / JavaScript served by Spring Boot
Browser	Any modern browser (Chrome, Edge, Firefox)
Version control	Git and GitHub

2.3 Development Tools and Frameworks

The backend was developed in Java 17 with Spring Boot 3.3.4 and built with Maven. Spring Data JPA (Hibernate) is used for the entities that the application writes directly, while Spring's `JdbcTemplate` is used to call the stored procedures and to read joined result sets. Flyway applies the three versioned SQL migrations that create the schema, the procedural logic and the seed data. MySQL Workbench is used to inspect the schema and the audit tables, and a REST client or `curl` can be used to exercise the API.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The user interface is a single static page (`src/main/resources/static/index.html`) written in plain HTML, CSS and JavaScript with no front-end framework or build step. Spring Boot serves it from the same port as the API, so the browser never deals with cross-origin configuration in normal use. The page contains the project overview sections, a public live-matching demo, and a donor dashboard that is revealed after login. It uses the native `<dialog>` element for the login and registration forms, the Fetch API for all HTTP calls, and `localStorage` to remember the logged-in session. Fonts (Manrope and Fraunces) are loaded from Google Fonts with system-font fallbacks.

3.2 Backend

The backend is a Spring Boot 3.3.4 REST API (Java 17) organised in the usual layered structure under `com.foooddonation:` `controller` (HTTP mapping), `service` (business orchestration), `repository` (Spring Data JPA), `entity` (JPA entities), `dto` (validated request/response objects), `security` (JWT), `config` (Spring Security configuration) and `exception` (uniform error handling). Because the matching rules live in MySQL, the service layer is deliberately thin: it validates input, calls the stored procedures, and reads back the results.

3.3 Database

MySQL 8 stores all persistent data in twelve tables. Referential integrity is enforced with foreign keys, uniqueness with UNIQUE constraints (one e-mail per user, one pickup per listing, unique locality and food-type names), and fast lookup of live listings with a composite B-tree index on `(status, expiry_time)`. The schema is created by Flyway migrations `V1__init_schema.sql`, `V2__triggers_procedures.sql` and `V3__seed_data.sql`; Hibernate is set to `ddl-auto=validate`, so it only checks that the entities match the schema and never alters it. The stored function, four stored procedures and four triggers described in Chapter 6 are part of the database, not the application.

3.4 Version Control

Git is used for version control, with the project kept in a single repository (Maven project root containing the source, the SQL migrations, the README and the SQL mapping reference) hosted on GitHub.

3.5 Authentication and Security Libraries

Library / Component	Purpose
Spring Security 6	Request authorization: <code>/api/auth/**</code> , <code>/api/match</code> , <code>/api/lookups</code> and static files are public; every other route requires authentication. Sessions are stateless; CSRF protection is disabled because no cookie-based session is used.
BCryptPasswordEncoder	Passwords are stored only as BCrypt hashes in <code>users.password_hash</code> .

Library / Component	Purpose
jjwt 0.12.5	Issues and verifies HS256-signed JSON Web Tokens whose subject is the user's e-mail; the lifetime is <code>app.jwt.expiration-ms</code> (24 hours by default).
Jakarta Bean Validation	DTO annotations (<code>@NotNull</code> , <code>@NotBlank</code> , <code>@Email</code> , <code>@Size</code> , <code>@DecimalMin</code>) reject malformed requests with HTTP 400 before any SQL runs.
CORS configuration	During local development, requests from <code>localhost / 127.0.0.1</code> origins are allowed.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The system follows a three-tier architecture that separates the user interface, the application logic and the persisted data. Figure 4.1 shows the layers and how a request flows between them.

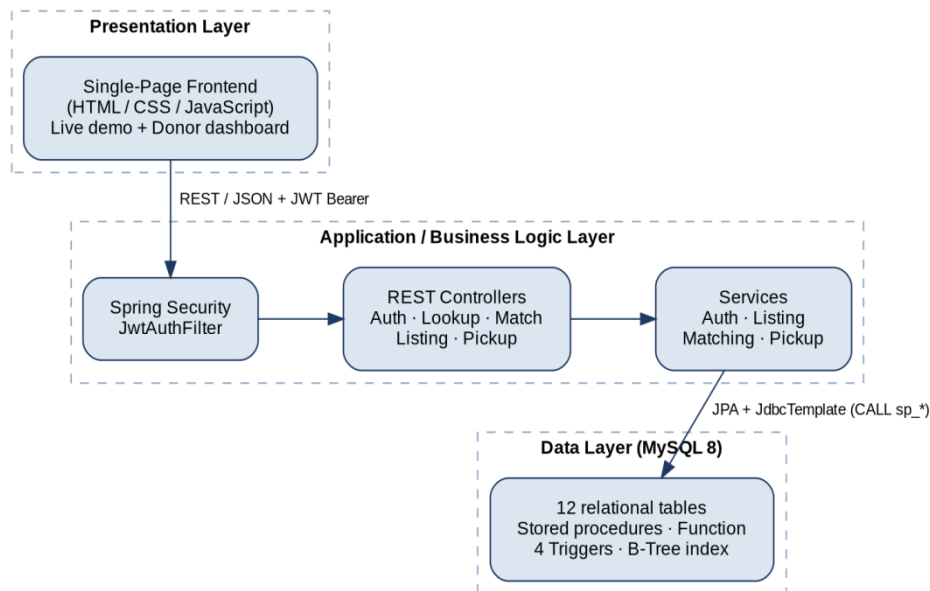


Figure 4.1: High-level three-tier architecture of the Food Donation and NGO Connect Platform

4.2 Description of Each Layer

Presentation layer. The single-page front end never talks to MySQL. Every piece of data it shows comes from the REST API over HTTP. After login, the JWT returned by the server is stored in the browser and sent as an `Authorization: Bearer` header with each protected request.

Application layer. Each request first passes through `JwtAuthFilter`, which validates the token and, if valid, marks the request as authenticated. The matching REST controller then validates the request body and delegates to a service. The services call stored procedures through `JDBC Template` (matching, pickup assignment, status updates) or save entities through `Spring Data JPA` (users, food listings). A global exception handler converts validation errors, business-rule errors raised inside MySQL with `SIGNAL SQLSTATE '45000'`, and unexpected failures into a uniform JSON error body with the correct HTTP status.

Data layer. MySQL holds both the data and the business rules that must never be bypassed: constraints, the distance function, the scoring procedure, the pickup-state machine and the triggers that keep related rows consistent. Connections are managed by Spring Boot's default HikariCP connection pool.

4.3 Deployment Architecture

For development and demonstration, the whole system runs on one machine as two processes: the Spring Boot application, which serves both the API and the static front end on

port 8080, and the MySQL server on its default port 3306. No containerisation or CI/CD pipeline was configured for this submission; Chapter 11 outlines how the same application could be moved to a hosted environment such as a cloud VM with AWS RDS or Azure SQL.

CHAPTER 5: DESIGN

5.1 Database Design Principles

The design was carried out in two phases. In the **conceptual phase**, the entities Donor, NGO, FoodListing, Volunteer and Pickup were identified together with Locality, FoodType and User, and their relationships and cardinalities were modelled in an Entity-Relationship diagram. In the **implementation phase**, the model was normalised to third normal form and written as DDL with primary keys, foreign keys, UNIQUE constraints and an index. Wherever possible, a rule is enforced inside the database — for example a quantity of zero or an expiry time in the past is rejected by a trigger even if the application layer is bypassed, and an illegal pickup status transition is rejected by a stored procedure.

5.2 ER Diagram / Database Schema

The database consists of twelve tables, shown in Figure 5.1. Blue tables hold core data, the grey table is the many-to-many junction, and the yellow tables are audit/notification tables that are written by triggers and stored procedures rather than by the application's JPA entities.

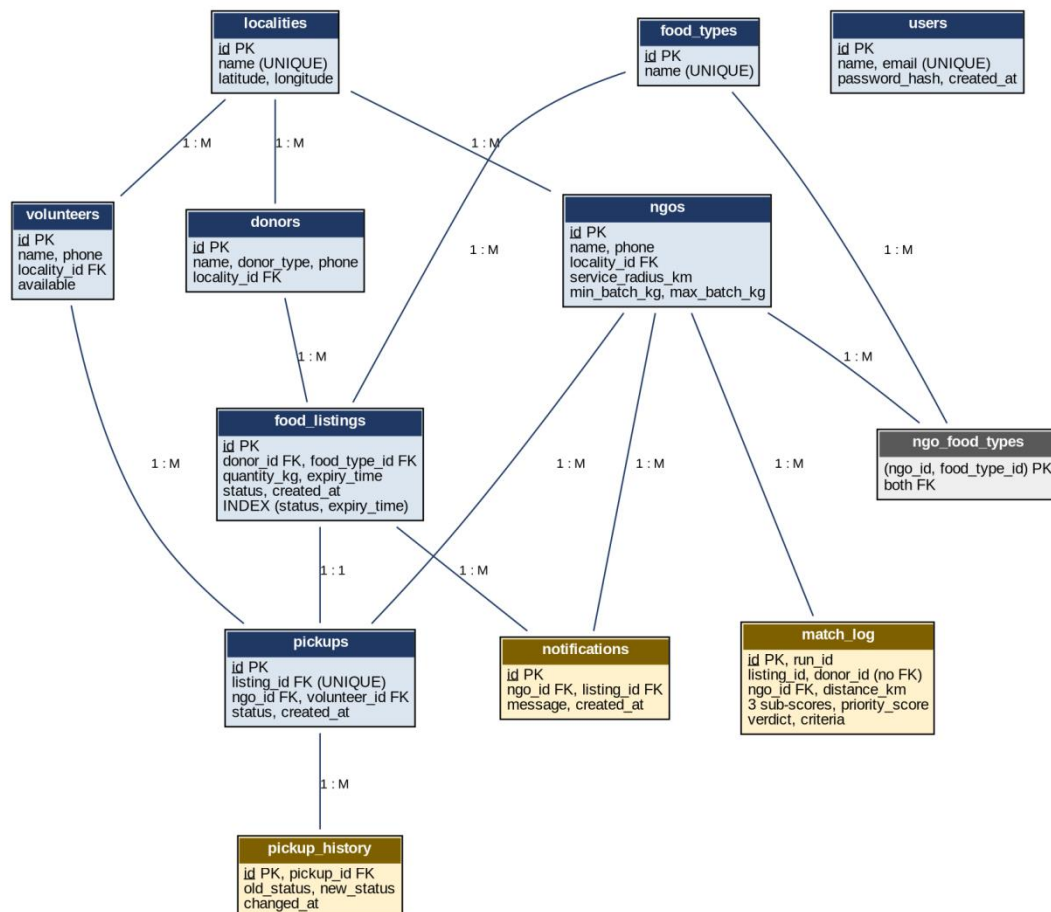


Figure 5.1: Entity-Relationship diagram of the 12-table database schema

Table summary

Table	Purpose	Key constraints
users	Login accounts (name, e-mail, BCrypt hash)	PK id; email UNIQUE

Table	Purpose	Key constraints
localities	Hyderabad localities with latitude/longitude centroids	PK id; name UNIQUE
food_types	Cooked meals, Bakery, Fresh produce, Packaged food, Dairy	PK id; name UNIQUE
donors	Restaurants, households and event organisers (donor_type)	FK locality_id → localities
ngos	NGOs with service radius and accepted batch range (min/max kg)	FK locality_id; defaults radius 10 km, batch 1–100 kg
ngo_food_types	Which food types each NGO accepts (many-to-many)	Composite PK (ngo_id, food_type_id); FKs; ON DELETE CASCADE from ngos
volunteers	Collectors with a locality and an available flag	FK locality_id
food_listings	A donation: donor, type, quantity (kg), expiry time, status	FKs donor/type; B-tree index (status, expiry_time)
pickups	The confirmed pairing of a listing with an NGO and a volunteer	listing_id UNIQUE (one pickup per listing); FKs listing/NGO/volunteer
pickup_history	Audit trail of every pickup status change (trigger-written)	FK pickup_id
notifications	Alerts to eligible NGOs about new listings (trigger-written)	FK ngo/listing; ON DELETE CASCADE from listings
match_log	Audit of every match candidate: scores, verdict and criteria	FK ngo_id; no FK on listing_id so the audit survives deletions; index on run_id

5.3 Normalization

The schema is in third normal form. All columns hold atomic values (1NF). Every non-key column depends on the whole of its table's key (2NF), and no non-key column depends on another non-key column (3NF). Concretely, locality coordinates are stored once in `localities` and referenced by donors, NGOs and volunteers; food-type names are stored once in `food_types`; the many-to-many relationship between NGOs and the food types they accept is resolved through the junction table `ngo_food_types`; and a listing's donor details are reached through the `donor_id` foreign key rather than copied into the listing. As a result no coordinate, name or phone number is repeated, and updating a locality's coordinates changes every dependent row at once.

5.4 Core DBMS Concepts Used

DBMS Concept	Implementation	Used For	Benefit
ER Modeling	Donor – NGO – FoodListing – Pickup entities	Conceptual schema design	Clear, unambiguous relationships
Normalization (3NF)	Decomposed donor / NGO / listing / locality tables	Eliminating redundancy	Consistent, update-safe data
Joins	Multi-table SQL joins	Matching donors to NGOs	Single query across listings and profiles
Triggers	BEFORE/AFTER INSERT and AFTER UPDATE triggers	Validation, NGO notification, status sync	Real-time, automatic updates

DBMS Concept	Implementation	Used For	Benefit
Stored procedures & functions	sp_match_candidates, sp_assign_pickup, fn_distance_km, ...	Encapsulating matching and state rules	Reusable, consistent rules
Indexing	B-tree index on (status, expiry_time)	Finding live, expiring listings	Logarithmic-time lookup instead of a full scan
Transactions & locking	SELECT ... FOR UPDATE inside procedures	Assigning a pickup / changing status	Prevents two users claiming the same listing

5.5 Design Decisions and Assumptions

- **Matching lives in MySQL.** The scoring rule is implemented in `sp_match_candidates` rather than in Java, so the rule is enforced next to the data and the same logic serves both the public demo and stored listings.
- **Locations are locality centroids.** Distances are computed between the latitude/longitude of the donor's locality and the NGO's locality (accuracy of roughly a kilometre). A production system would use real geocoding.
- **Travel-time assumption.** An NGO is considered able to reach the food only if $\text{hours_to_expiry} \times 60 \geq \text{distance} \times 2.4 + 30$, i.e. travel at about 25 km/h plus 30 minutes of handling time.
- **Audit rows have no foreign key to listings.** `match_log.listing_id` and `donor_id` are plain columns (NULL for ad-hoc demo runs) so audit evidence survives if a listing is deleted.
- **One pickup per listing.** A UNIQUE constraint on `pickups.listing_id` makes double-assignment impossible at the database level.
- **All timestamps are UTC.** The JDBC URL sets `connectionTimeZone=UTC`, and Java and SQL both use UTC, so expiry calculations agree regardless of where the server runs.
- **Status values** (AVAILABLE, MATCHED, PICKED_UP, DELIVERED) are stored as VARCHAR and their transitions are controlled by the procedures and triggers, rather than by a CHECK constraint or lookup table.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

Module	Classes	Responsibility
Authentication	AuthController, AuthService, JwtService, JwtAuthFilter, User, UserRepository	Register (e-mail normalised to lower case, duplicate e-mail rejected with 409, BCrypt hash stored), login (401 on bad credentials) and JWT verification on every protected request
Lookup	LookupController	Public reference data (localities, food types, donors, NGOs) for the dropdowns
Matching	MatchController, MatchingService, MatchRequest, MatchResponse	Calls <code>sp_match_candidates</code> (ad-hoc demo) or <code>sp_match_listing</code> (stored listing) with a fresh run UUID, then reads the ranked rows back from <code>match_log</code> and builds a headline (Verified match / Needs review / Weak / No match)
Listings	ListingController, ListingService, FoodListing, FoodListingRepository	Inserts a listing through JPA (the triggers validate it and notify NGOs), converts “hours until expiry” to a UTC expiry time, returns the ranked matches, lists the latest 50 listings with pickup/NGO/volunteer, and deletes only AVAILABLE listings
Pickups	PickupController, PickupService	Calls <code>sp_assign_pickup</code> , <code>sp_update_pickup_status</code> and reads <code>pickup_history</code>
Error handling	GlobalExceptionHandler, ApiException	Uniform JSON errors; MySQL <code>SQLSTATE 45000</code> messages become HTTP 400 with the original message

6.2 Matching Engine (Stored Procedures)

The core of the system is the stored procedure `sp_match_candidates`, which uses the stored function `fn_distance_km` (the Haversine great-circle distance in kilometres). Figure 6.1 summarises the pipeline; Table 6.1 lists the database routines.

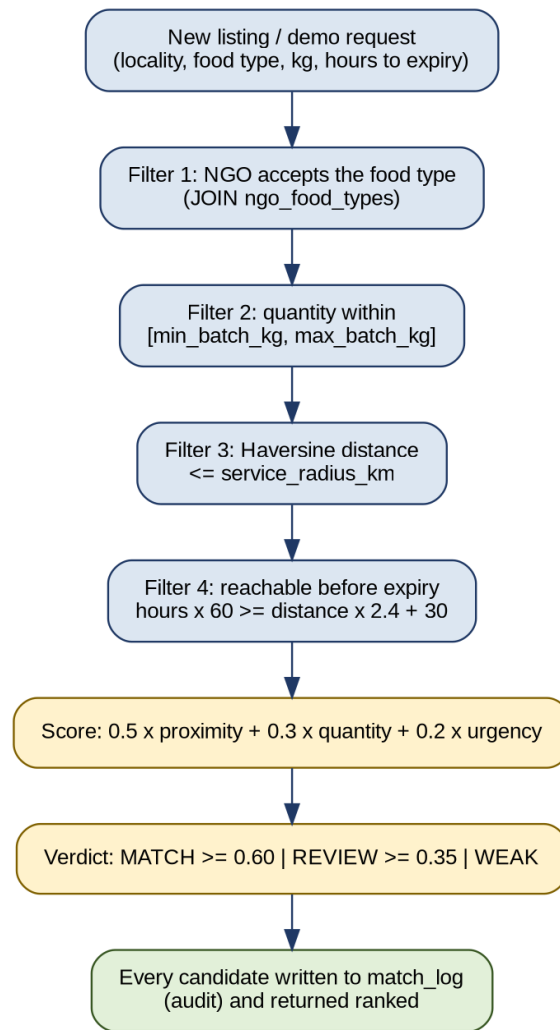


Figure 6.1: Matching pipeline inside MySQL (*sp_match_candidates*)

Routine	Type	What it does
<code>fn_distance_km</code>	Function	Haversine distance between two latitude/longitude pairs, returned in km
<code>sp_match_candidates</code>	Procedure	Applies the four hard filters, computes the three sub-scores and the priority score, assigns the verdict, and inserts every candidate into <code>match_log</code> under a run id
<code>sp_match_listing</code>	Procedure	Loads a stored AVAILABLE listing (donor locality, food type, quantity, remaining hours) and calls <code>sp_match_candidates</code>
<code>sp_assign_pickup</code>	Procedure	Locks the listing (FOR UPDATE), rejects it if missing or not AVAILABLE, picks the nearest available volunteer, and inserts the pickup
<code>sp_update_pickup_status</code>	Procedure	Locks the pickup and allows only ASSIGNED → PICKED_UP and PICKED_UP → DELIVERED; anything else raises “Invalid status transition.”

Table 6.1: Stored function and procedures

Matching rule

An NGO becomes a candidate only if **all** of the following hold: (1) it accepts the listing’s food type; (2) the quantity lies within its `[min_batch_kg, max_batch_kg]` range; (3) the

distance is within its `service_radius_km`; and (4) the food can be reached before it expires. Each candidate is then scored:

```
priority = 0.5 * proximity + 0.3 * quantity + 0.2 * urgency
proximity = 1 - distance / service_radius      (clamped at 0)
quantity = min(1, qty / max_batch_kg)
urgency = 1 - hours_to_expiry / 24             (clamped to 0..1)
verdict = MATCH if priority >= 0.60, REVIEW if >= 0.35, otherwise WEAK
```

Worked example

The following example applies the above rules to the sample data in `v3__seed_data.sql`. It is a hand calculation that follows the stored procedure's logic: a restaurant in Kukatpally offers **20 kg of cooked meals with 6 hours to expiry**.

NGO	Distance (km)	Proximity	Quantity	Urgency	Priority	Verdict
Hope Meals Trust	0.000	1.000	0.133	0.750	0.690	MATCH
Sunrise Shelter	4.757	0.524	0.333	0.750	0.512	REVIEW
City Care Kitchen	8.202	0.453	0.100	0.750	0.407	REVIEW
Little Hands Foundation	6.624	0.172	0.500	0.750	0.386	REVIEW
Anna Seva Society	8.119	0.188	0.250	0.750	0.319	WEAK

Hope Meals Trust ranks first because the donor and the NGO are in the same locality (proximity 1.0). In a second example, 30 kg of cooked meals offered from Quthbullapur with 3 hours to expiry, Sunrise Shelter (11.25 km) and Anna Seva Society (14.12 km) are excluded because they lie outside their service radii, and Little Hands Foundation ranks first with a priority of 0.900.

6.3 Triggers

Trigger	Event	Effect
<code>trg_listing_before_insert</code>	BEFORE INSERT on <code>food_listings</code>	Rejects <code>quantity ≤ 0</code> and expiry times that are not in the future (<code>SIGNAL SQLSTATE '45000'</code> , surfaced as HTTP 400)
<code>trg_listing_after_insert</code>	AFTER INSERT on <code>food_listings</code>	Inserts a notifications row for every NGO that accepts the food type, whose batch range fits the quantity and whose service radius covers the donor's locality
<code>trg_pickup_after_insert</code>	AFTER INSERT on <code>pickups</code>	Sets the listing to MATCHED, marks the volunteer unavailable, writes the first <code>pickup_history</code> row
<code>trg_pickup_after_update</code>	AFTER UPDATE on <code>pickups</code>	On a status change: logs history, copies the status to the listing, and frees the volunteer when the pickup is DELIVERED



Figure 6.2: Donation lifecycle and the database objects that drive each transition

6.4 Front-end Logic

The front end is organised around a small set of JavaScript functions. `loadLookups()` fills the dropdowns from `/api/lookups`; `runMatch()` posts the demo form to `/api/match` and renders the ranked candidates with their sub-scores and criteria text; `handleRegister()` and `handleLogin()` store the returned token and name in `localStorage` and reveal the dashboard; `postListing()` creates a donation and displays the ranked matches; `confirmPickup()`, `setStatus()` and `showHistory()` drive the pickup lifecycle; and `loadListings()` renders the latest listings with context-sensitive action buttons (Find NGOs / Remove for AVAILABLE listings, Mark picked up for ASSIGNED, Mark delivered for PICKED_UP, History once a pickup exists). All user-supplied text is HTML-escaped before it is inserted into the page, and errors are shown as toast messages.

6.5 API Integration

Two small helpers wrap `fetch`: `apiPost()` for public calls, and `authFetch()` for protected calls, which attaches the stored JWT as a Bearer token and parses the JSON error body so the user sees the server's message (for example "Listing is no longer available."). No page hard-codes a URL more than once, and every protected action goes through the same helper.

6.6 Authentication and Authorization

Registration creates a user with a BCrypt-hashed password and returns a JWT immediately; login verifies the hash and returns a JWT. `JwtAuthFilter` runs before Spring's username/password filter, validates the `Authorization: Bearer` token, and places an authenticated principal (the e-mail) into the security context. The security configuration then permits only the authentication endpoints, the public demo (`/api/match`), the lookup data and the static files without a token. In the current version authentication is all-or-nothing: any logged-in user may act on any listing, and there is not yet a separate donor, NGO or volunteer role (see Section 10.3 and Chapter 11).

CHAPTER 7: FEATURES

7.1 List of Main Features

- Public **live matching demo**: choose a locality, food type, quantity and hours to expiry and run the matching engine without logging in.
- Transparent ranking: every candidate NGO is shown with distance, proximity / quantity / urgency scores, a priority score, a verdict and the criteria that qualified it.
- User registration and login with BCrypt-hashed passwords and JWT sessions.
- Donor dashboard: post a donation, see ranked NGO matches immediately, and manage all recent listings.
- Automatic notification of every eligible NGO when a valid listing is posted.
- One-click **pickup confirmation** that assigns the nearest available volunteer.
- Step-by-step status tracking (ASSIGNED → PICKED_UP → DELIVERED) with an enforced order.
- Per-pickup **history** (audit trail) and a database-level `match_log` of every candidate considered.
- Re-matching of an AVAILABLE listing (“Find NGOs”) and removal of listings that have not been matched yet.
- Business-rule validation in the database: positive quantity, future expiry, one pickup per listing, valid status transitions.

7.2 How Each Feature Works

Live matching demo. The form values are posted to `POST /api/match`. `MatchingService` generates a run UUID and calls `sp_match_candidates` with a NULL listing and donor, so nothing is stored as a listing; the candidates are written to `match_log` under the run id and read back ordered by priority score. The response contains a verdict (MATCH, REVIEW, WEAK or NO_MATCH), a headline and a summary such as “5 eligible NGO(s). Best: Hope Meals Trust (0.000 km, priority 0.690).”

Posting a donation. `POST /api/listings` converts “hours until expiry” into a UTC expiry timestamp and saves the listing. The BEFORE INSERT trigger rejects invalid values; the AFTER INSERT trigger writes one notification per eligible NGO (for example “New Cooked meals donation: 20.00 kg, 4.8 km away”). The service then calls `sp_match_listing` and returns the new listing id together with the ranked matches.

Confirming a pickup. The dashboard sends `{listingId, ngoId}` to `POST /api/pickups`. `sp_assign_pickup` locks the listing row, confirms it is still AVAILABLE, chooses the available volunteer whose locality is nearest to the donor’s locality, and inserts the pickup. The AFTER INSERT trigger then marks the listing MATCHED, makes the volunteer unavailable and opens the history.

Tracking delivery. `PUT /api/pickups/{id}/status` calls `sp_update_pickup_status`, which permits only the two forward transitions. The AFTER UPDATE trigger records the change in `pickup_history`, synchronises the listing status and releases the volunteer on DELIVERED. `GET /api/pickups/{id}/history` returns the trail.

Deleting a listing. `DELETE /api/listings/{id}` removes a listing only while it is AVAILABLE (otherwise 409). Its notifications are removed by `ON DELETE CASCADE`, while its `match_log` rows remain as audit evidence.

Application screenshots

Screenshots of the running application are to be inserted below before submission (see also Appendix E).

Figure 7.1: Live matching demo with ranked NGO candidates

NGO Connect Overview Methodology DBMS Tech & Concepts Live Demo Team \$ sathwika Log out

PICKUP LOCALITY: Outhbullapur FOOD TYPE: Cooked meals QUANTITY (KG): 45 HOURS UNTIL EXPIRY: 4

Run matching engine

● **Candidates need a quick review**
2 eligible NGO(s). Best: Hope Meals Trust (sample) (6.624 km, priority 0.481).

NGO Name	Proximity	Review Score
Hope Meals Trust (sample)	Kukatpally · 6.6 km	REVIEW 0.48
City Care Kitchen (sample)	Ameerpet · 8.2 km	REVIEW 0.46

$$\text{priority} = 0.5 \cdot \text{proximity} + 0.3 \cdot \text{quantity} + 0.2 \cdot \text{urgency} \cdot \text{MATCH} \geq 0.60 \cdot \text{REVIEW} \geq 0.35$$

Figure 7.2: Donor dashboard – post a donation and confirm a pickup

Post donation

Listing #5 – No eligible NGO right now

No NGO accepts this food type within its service radius and batch range while the food is still fresh enough to reach it. The listing stays AVAILABLE and can be re-matched.

DONOR	FOOD	KG	EXPIRES	STATUS	NGO / VOLUNTEER	
Grand Hall Events (sample)	Bakery	10	10/2/26, 7:38 PM	AVAILABLE	—	Find NGOs Remove
Crust & Crumb Bakery (sample)	Cooked meals	20	10/1/26, 4:38 PM	DELIVERED	Hope Meals Trust (sample) Imran (sample)	History

CHAPTER 8: TESTING

8.1 API Testing Approach

The REST API consists of eleven endpoints (Appendix D). Each endpoint is tested for the normal case and for the failure cases that its validation and business rules are designed to catch. Requests are sent with a REST client such as Postman or with `curl`; the login or registration response supplies the JWT, which is then sent as `Authorization: Bearer <token>` to the protected endpoints. The expected status codes follow from the code: DTO validation failures return 400, business-rule errors raised in MySQL return 400 with the message from the procedure or trigger, a duplicate e-mail returns 409, bad credentials return 401, a missing listing returns 404, and deleting a non-AVAILABLE listing returns 409.

8.2 Database-level Testing

Because much of the business logic lives in MySQL, the procedures and triggers are also tested directly with the MySQL client or Workbench, independently of Spring Boot. This verifies that the rules hold even when the API layer is bypassed: for example, inserting a listing with quantity 0 or a past expiry time directly into `food_listings` must raise the trigger's error, and calling `sp_update_pickup_status` with an illegal transition must raise "Invalid status transition.". After each scenario the audit tables (`match_log`, `notifications`, `pickup_history`) are queried to confirm the expected rows were written.

```
USE fooddonation;
SELECT * FROM food_listings;
SELECT * FROM notifications ORDER BY id DESC;
SELECT * FROM match_log ORDER BY id DESC LIMIT 10;
SELECT * FROM pickup_history;
```

8.3 Tools Used for Testing

- Postman or `curl` – request/response testing of the REST endpoints.
- MySQL command-line client / MySQL Workbench – direct testing of triggers and procedures and inspection of table contents.
- A web browser – manual testing of the live demo and the donor dashboard.
- Spring Boot start-up with Flyway – each fresh run applies V1–V3 from scratch, which doubles as a check that the migrations are valid.

8.4 Test Cases and Results

Table 8.1 lists the test cases derived from the system's rules. The expected results follow from the implemented code and seed data; the final column is to be completed with the outcome observed on the running system.

#	Test Case	Expected Result	Actual Result
1	Register with a new e-mail and a valid password	200; JWT returned; <code>users.password_hash</code> holds a BCrypt hash, not the password	☐ Pass ☐ Fail
2	Register again with the same e-mail (any letter case)	409 "An account with that email already exists."	☐ Pass ☐ Fail
3	Register with a 4-character password	400 "Password must be at least 6 characters."	☐ Pass ☐ Fail
4	Login with a wrong password	401 "Invalid email or password."	☐ Pass ☐ Fail

#	Test Case	Expected Result	Actual Result
5	Call GET /api/listings without a token	Request rejected (not authenticated)	☐ Pass ☐ Fail
6	Call GET /api/lookups without a token	200; localities, food types, donors, NGOs returned	☐ Pass ☐ Fail
7	Demo match: Kukatpally, Cooked meals, 20 kg, 6 h	5 candidates; Hope Meals Trust first, priority 0.69, verdict MATCH (see Section 6.2)	☐ Pass ☐ Fail
8	Demo match: Quthbullapur, Cooked meals, 30 kg, 3 h	Sunrise Shelter and Anna Seva Society excluded (outside radius); Little Hands Foundation first, priority 0.90	☐ Pass ☐ Fail
9	Demo match with 500 kg	Verdict NO_MATCH, “No eligible NGO right now” (above every NGO’s max batch)	☐ Pass ☐ Fail
10	Demo match with quantity 0 or hours 0.2	400 “Quantity must be at least 0.1 kg.” / “Hours until expiry must be at least 0.5.”	☐ Pass ☐ Fail
11	Post a valid listing	200 with listingId and ranked matches; one notifications row per eligible NGO	☐ Pass ☐ Fail
12	Insert a listing with quantity 0 directly in SQL	Error “Quantity must be greater than zero.”	☐ Pass ☐ Fail
13	Insert a listing with a past expiry directly in SQL	Error “Expiry time must be in the future.”	☐ Pass ☐ Fail
14	Confirm a pickup for an AVAILABLE listing	Pickup ASSIGNED; listing MATCHED; nearest free volunteer assigned and set unavailable; first history row written	☐ Pass ☐ Fail
15	Confirm a pickup for the same listing again	400 “Listing is no longer available.”	☐ Pass ☐ Fail
16	Confirm a pickup for a non-existent listing id	400 “Listing not found.”	☐ Pass ☐ Fail
17	Set status DELIVERED directly from ASSIGNED	400 “Invalid status transition.”	☐ Pass ☐ Fail
18	Set PICKED_UP, then DELIVERED	204 each; listing status follows; volunteer available again; history has 3 rows	☐ Pass ☐ Fail
19	Delete an AVAILABLE listing / delete a MATCHED listing	204 / 409 “Only AVAILABLE listings can be removed.”	☐ Pass ☐ Fail
20	Delete a non-existent listing	404 “Listing not found.”	☐ Pass ☐ Fail
21	After deleting a listing, query match_log	Its audit rows are still present (no FK on listing_id)	☐ Pass ☐ Fail

Table 8.1: Test cases

8.5 System Performance Analysis

The project presentation reports two headline outcomes of the platform compared with manual coordination: an approximately **82% reduction in donor-to-NGO matching time**, and a **65% reduction in unclaimed surplus food** after clustering donations by locality. These figures are quoted from the presentation; the measurement setup behind them (sample size, baseline and period) should be stated alongside them when the report is submitted. On the technical side, candidate retrieval is a single set-based SQL statement executed inside the database, and listings are looked up through the (status, expiry_time) B-tree index, so matching cost grows with the number of NGOs rather than with the number of historical listings.

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

1. Install Java 17+, Maven 3.9+ and MySQL 8, and make sure MySQL is running on port 3306.
2. Open `src/main/resources/application.properties` and set `spring.datasource.username` and `spring.datasource.password` for your MySQL account.
3. From the project root run `mvn spring-boot:run`. On start-up Flyway creates the `fooddonation` database (the JDBC URL has `createDatabaseIfNotExist=true`) and applies V1 (schema), V2 (function, procedures, triggers) and V3 (seed data).
4. Open `http://localhost:8080` in a browser. Try the live demo, or register an account and use the dashboard.
5. Verify the data in MySQL with the queries in Section 8.2.

9.2 Environment Configuration

Property	Purpose	Default / note
<code>spring.datasource.url</code>	JDBC URL; sets UTC time zone and creates the database if absent	JDBC :mysql://localhost:3306/fooddonation?...connectionTimeZone=UTC
<code>spring.datasource.username/password</code>	MySQL credentials	root / placeholder – change to your own
<code>spring.jpa.hibernate.ddl-auto</code>	Hibernate only validates the Flyway-created schema	validate
<code>spring.flyway.locations</code>	Location of migrations	classpath:db/migration
<code>app.jwt.secret</code>	HMAC key used to sign JWTs	Placeholder – must be replaced with a long random value before any real deployment
<code>app.jwt.expiration-ms</code>	Token lifetime	86400000 (24 hours)

Property	Purpose	Default / note
<code>server.port</code>	HTTP port	8080

9.3 Hosting of Frontend and Backend

The front end is bundled inside the Spring Boot application, so there is one deployable unit and one port. For this submission the application and the database run locally, which is appropriate for development and in-class demonstration. For a hosted deployment, the application can be packaged with `mvn package` into a single runnable JAR and run on a cloud VM or container platform, with MySQL provided by a managed service such as AWS RDS or Azure SQL. Before doing so the JWT secret must be replaced, the CORS origins in `SecurityConfig` tightened to the real front-end origin, and database credentials supplied through environment variables rather than committed in `application.properties`.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- **Where the matching rule should live.** Implementing the scoring in Java would have been simpler to write but would have split the rules between two places. The team had to design the logic as set-based SQL inside stored procedures.
- **Consistency between related tables.** A pickup changes the pickup row, the listing row and the volunteer row. Doing this from application code risks leaving them out of step if one update fails.
- **Time-zone consistency.** Expiry checks compare a timestamp written by Java with `UTC_TIMESTAMP()` evaluated inside MySQL; unless both use UTC, listings can appear expired or immortal depending on server settings.
- **Surfacing database rule violations as friendly API errors.** Errors raised by triggers and procedures arrive in Java as low-level SQL exceptions.
- **Running procedural SQL through Flyway.** Triggers, functions and procedures contain semicolons and need a `DELIMITER` block, and creating them requires sufficient MySQL privileges.
- **Concurrent confirmation.** Two users could try to confirm the same listing at the same moment.

10.2 Solutions Applied

- The scoring and filtering were written once in `sp_match_candidates`, with `fn_distance_km` as a reusable function; the Java service only calls the procedure and reads `match_log`. This also gave a free audit trail.
- State changes are driven by AFTER INSERT / AFTER UPDATE triggers, so the listing, pickup, volunteer and history rows always change together inside the same database transaction.
- All timestamps are stored in UTC: the JDBC URL sets `connectionTimeZone=UTC` with `forceConnectionTimeZoneToSession=true`, and Java uses `LocalDateTime.now(ZoneOffset.UTC)`.
- Procedures and triggers raise `SIGNAL SQLSTATE '45000'` with a human-readable message, and `GlobalExceptionHandler` maps that `SQLSTATE` to HTTP 400 with the same message.
- The routines are delivered as a versioned Flyway migration (v2), and the README documents that the migration user needs trigger/function privileges.
- `sp_assign_pickup` and `sp_update_pickup_status` lock the row with `SELECT ... FOR UPDATE`, and `pickups.listing_id` is `UNIQUE`, so a second confirmation fails with “Listing is no longer available.”

10.3 Current Limitations or Known Bugs

- **No role model.** Any logged-in user can act on any listing; there are no separate donor, NGO or volunteer roles or ownership checks, and `JwtAuthFilter` grants no authorities.

- **Open demo endpoint.** `POST /api/match` is public by design but writes candidate rows to `match_log` for anonymous requests.
- **`sp\_assign\_pickup` does not verify the NGO.** It does not check that the chosen NGO was an eligible candidate for the listing, and if no volunteer is available the pickup is created with no volunteer assigned.
- **Coarse locations.** Distances use locality centroids (about 1 km accuracy) and a fixed 25 km/h travel-speed assumption.
- **Notifications are database rows only.** No e-mail, SMS or push message is sent, and there is no NGO login to view them.
- **No automatic expiry.** A listing whose expiry time has passed is not moved to an “EXPIRED” state automatically.
- **Development-grade security settings.** The default JWT secret and the localhost CORS list are suitable for local use only; there is no refresh-token or password-reset flow.
- **No automated test suite.** The Maven project includes the Spring test dependencies, but no unit or integration tests have been written yet.
- **Single-node deployment.** The system has been run only on a local machine, not on a hosted environment.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- **Role-based access:** separate donor, NGO and volunteer accounts, with ownership checks so users can act only on their own listings, pickups and notifications.
- **Predictive demand:** forecasting NGO food demand by season and locality from historical donation and request data.
- **Verified trust network:** ratings and verification of donors and NGOs, building a trusted network of recurring participants.
- **Multi-city expansion:** extending the schema and matching logic to onboard NGOs and donors across cities and languages.
- **Real notifications:** e-mail, SMS or push delivery of the notification rows, and an NGO dashboard to accept or decline offers.
- **Automatic expiry:** a scheduled MySQL event that marks listings EXPIRED once their time has passed.
- **Mobile applications** for donors, NGOs and volunteers.

11.2 Possible Integrations or Optimizations

- Real geocoding and routing (for example a maps API) to replace locality centroids and the fixed travel-speed assumption.
- Spatial indexes (MySQL `POINT` columns with `SPATIAL INDEX`) to prune distant NGOs before computing Haversine distances.
- Managed database hosting (AWS RDS or Azure SQL) and containerisation with Docker for reproducible deployment.
- An automated test suite (JUnit/Spring Boot Test with Testcontainers for MySQL) that exercises the procedures and triggers on every build.
- Caching of reference data (localities, food types) and pagination of the listings endpoint.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

This project set out to replace slow, informal coordination of surplus-food donations with a structured, database-driven platform. A twelve-table, third-normal-form MySQL schema, a stored function and four stored procedures, four triggers, a Spring Boot REST API secured with JWT, and a single-page web interface were designed and built so that a donation can be posted, matched, confirmed, collected and delivered with every step recorded.

12.2 What Was Achieved

- A transparent matching engine inside MySQL that filters by food type, distance, quantity and shelf life, scores candidates with a documented formula, and logs every candidate.
- Automatic validation of new listings and automatic notification of eligible NGOs through triggers.
- A pickup lifecycle with an enforced state order, nearest-volunteer assignment, protection against double assignment, and a full status history.
- Secure user accounts with BCrypt password hashing and stateless JWT authentication.
- A public live demonstration and a donor dashboard that make the whole process visible end to end.

12.3 Skills Learned

The project gave practical experience in ER modelling and normalisation to 3NF; writing DDL with keys, constraints and indexes; writing stored functions, stored procedures and triggers in MySQL, including `SIGNAL`-based business-rule errors and row locking; managing schema versions with Flyway; building a layered Spring Boot REST API with Spring Data JPA, `JdbcTemplate`, Spring Security and JWT; handling time zones consistently across Java and SQL; and presenting a database design clearly through diagrams and documentation.

CHAPTER 13: REFERENCES

- React — Official Documentation, <https://react.dev/>
- Vite — Official Documentation, <https://vitejs.dev>
- Node.js — Official Documentation, <https://nodejs.org/en/docs>
- Express.js — Official Documentation, <https://expressjs.com>
- MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en/>
- jsonwebtoken — npm documentation, <https://www.npmjs.com/package/jsonwebtoken>
- bcrypt — npm documentation, <https://www.npmjs.com/package/bcrypt>
- Postman Learning Center, <https://learning.postman.com>
- MDN Web Docs — JavaScript and Web APIs reference, <https://developer.mozilla.org/>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippets

A.1 Haversine distance function (V2__triggers_procedures.sql)

```
CREATE FUNCTION fn_distance_km(lat1 DECIMAL(9,6), lon1 DECIMAL(9,6),
                             lat2 DECIMAL(9,6), lon2 DECIMAL(9,6))
RETURNS DECIMAL(8,3) DETERMINISTIC
BEGIN
    RETURN 6371 * 2 * ASIN(SQRT(
        POWER(SIN(RADIANS(lat2 - lat1) / 2), 2) +
        COS(RADIANS(lat1)) * COS(RADIANS(lat2)) *
        POWER(SIN(RADIANS(lon2 - lon1) / 2), 2)));
END
```

A.2 Hard filters in sp_match_candidates (excerpt)

```
FROM ngos n
JOIN ngo_food_types nft ON nft.ngo_id = n.id AND nft.food_type_id = p_food_type
JOIN localities nl ON nl.id = n.locality_id
JOIN localities src ON src.id = p_locality
WHERE p_qty BETWEEN n.min_batch_kg AND n.max_batch_kg
...
WHERE b.d <= b.radius
AND p_hours * 60 >= b.d * 2.4 + 30 -- ~25 km/h travel + 30 min handling
```

A.3 Trigger that notifies eligible NGOs

```
CREATE TRIGGER trg_listing_after_insert AFTER INSERT ON food_listings FOR EACH ROW
BEGIN
    INSERT INTO notifications (ngo_id, listing_id, message)
    SELECT n.id, NEW.id, CONCAT('New ', ft.name, ' donation: ', NEW.quantity_kg, ' kg,
    ',
        ROUND(fn_distance_km(dl.latitude, dl.longitude,
        nl.latitude, nl.longitude), 1), ' km away')
    FROM ngos n
    JOIN ngo_food_types nft ON nft.ngo_id = n.id AND nft.food_type_id =
NEW.food_type_id
    ... -- joins to food_types, localities and donors
    WHERE NEW.quantity_kg BETWEEN n.min_batch_kg AND n.max_batch_kg
    AND fn_distance_km(dl.latitude, dl.longitude,
        nl.latitude, nl.longitude) <= n.service_radius_km;
END
```

A.4 Java service calling the stored procedure (MatchingService)

```
public MatchResponse matchListing(Long listingId) {
    String run = UUID.randomUUID().toString();
    jdbc.update("CALL sp_match_listing(?,?)", run, listingId);
    return describe(run); // reads ranked rows back from match_log
}
```

Appendix B: Installation and Setup Instructions

6. Prerequisites: Java 17+, Maven 3.9+, MySQL 8 running locally.
7. Edit `src/main/resources/application.properties` to set your MySQL username and password.
8. Run `mvn spring-boot:run` from the project root (Flyway applies V1, V2 and V3 automatically).
9. Open `http://localhost:8080` in a browser.

Appendix C: Seed Data and Accounts

The migration `v3__seed_data.sql` does not create any user accounts — register a new account from the web page to log in. It does load fictional sample data (marked “(sample)”) so that the matching demo works on first run:

Entity	Seeded records
Localities (10)	Quthbullapur, Bachupally, Kukatpally, Miyapur, Kondapur, Madhapur, Gachibowli, Jubilee Hills, Ameerpet, Secunderabad
Food types (5)	Cooked meals, Bakery, Fresh produce, Packaged food, Dairy
Donors (5)	Spice Route Restaurant, Green Leaf Caterers, Sharma Household, Crust & Crumb Bakery, Grand Hall Events
NGOs (5)	Hope Meals Trust, Sunrise Shelter, Little Hands Foundation, City Care Kitchen, Anna Seva Society
Volunteers (4)	Ravi, Meera, Imran, Latha

Appendix D: API Reference Summary

Method	Path	Auth	Purpose
POST	/api/auth/register	No	Create account; returns JWT
POST	/api/auth/login	No	Log in; returns JWT
GET	/api/lookups	No	Localities, food types, donors, NGOs
POST	/api/match	No	Rank NGOs for an ad-hoc donation (logged to <code>match_log</code>)
GET	/api/listings	Yes	Latest 50 listings with pickup / NGO / volunteer
POST	/api/listings	Yes	Create listing (trigger notifies NGOs); returns ranked matches
GET	/api/listings/{id}/matches	Yes	Re-run matching for an AVAILABLE listing
DELETE	/api/listings/{id}	Yes	Remove an AVAILABLE listing
POST	/api/pickups	Yes	Confirm { <code>listingId</code> , <code>ngoId</code> }; assigns nearest free volunteer
PUT	/api/pickups/{id}/status	Yes	Set PICKED_UP, then DELIVERED
GET	/api/pickups/{id}/history	Yes	Audit trail of status changes

Appendix E: Screenshots

The following screenshots are to be taken from the running application and inserted before submission, in this order:

1. Home page / abstract section

The screenshot shows the home page of the NGO Connect application. The header includes the logo, navigation links (Overview, Methodology, DBMS, Tech & Hardware, Live Demo, Team), and user actions (Log in, Register). A secondary navigation bar lists specific topics: Overview, Methodology, DBMS, Concepts, Tech & Hardware, Live Demo, and Team.

The main content area features a large heading "No surplus food goes unclaimed." followed by a paragraph explaining the platform's purpose: "Restaurants and households discard edible surplus daily while NGOs struggle to find nearby donors in time. This platform stores donor listings and NGO requirements in a relational database and connects them through SQL joins, triggers and stored procedures."

Below the text are two buttons: "Try the matching engine" and "Read the methodology". To the right, a box displays "120 kg cooked meals" from a "Restaurant cluster - daily surplus" and links to "Nearby NGOs" (Shelters reporting shortages). A "VERIFIED MATCH" section highlights "St. Anthony's Shelter" as "matched to 3 donors - 45 kg secured today".

At the bottom, two statistics are shown: "-82% faster donor-to-NGO matching vs. manual" and "65% less unclaimed surplus after locality clustering".

2. Live matching demo showing ranked NGO candidates

The screenshot displays the live matching demo interface. The header is consistent with the home page, but the user is logged in as "sathwika".

The main form allows users to input search criteria: "PICKUP LOCALITY" (Outhbullapur), "FOOD TYPE" (Cooked meals), "QUANTITY (KG)" (45), and "HOURS UNTIL EXPIRY" (4). A "Run matching engine" button is positioned to the right of the form.

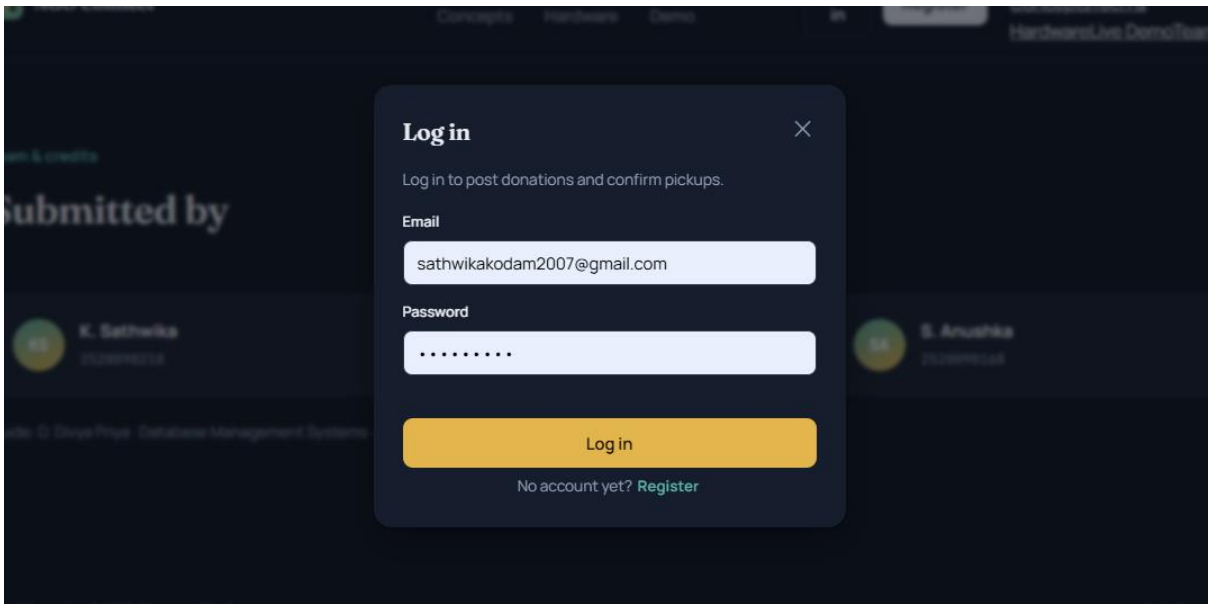
Below the form, a section titled "Candidates need a quick review" indicates "2 eligible NGO(s). Best: Hope Meals Trust (sample) (6.624 km, priority 0.481)".

Two candidate cards are shown:

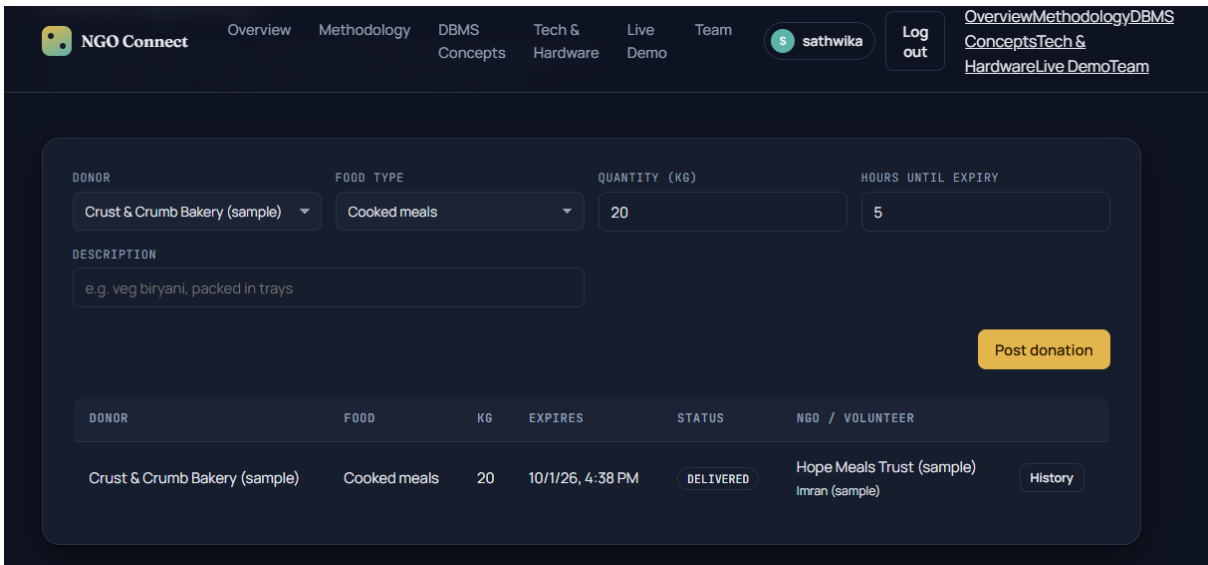
- Hope Meals Trust (sample)**: Kukatpally - 6.6 km. A progress bar is shown, and a "REVIEW 0.48" button is available.
- City Care Kitchen (sample)**: Ameerpet - 8.2 km. A progress bar is shown, and a "REVIEW 0.46" button is available.

At the bottom, a formula for the priority score is displayed: $\text{priority} = 0.5 \cdot \text{proximity} + 0.3 \cdot \text{quantity} + 0.2 \cdot \text{urgency} \cdot \text{MATCH} \geq 0.60 \cdot \text{REVIEW} \geq 0.35$.

3. Register and Log-in dialogs



4. Donor dashboard – posting a donation with ranked matches



5. Pickup confirmed – volunteer assigned

NGO Connect OverviewMethodologyDBMS ConceptsTech & HardwareLive DemoTeam s sathwika Log out OverviewMethodologyDBMS ConceptsTech & HardwareLive DemoTeam						
Crust & Crumb Bakery (sample)	Cooked meals	20	10/1/26, 4:38 PM	DELIVERED	Hope Meals Trust (sample) Imran (sample)	History

6. MySQL Workbench – EER diagram / table list of the 12 tables

NGO Connect

OverviewMethodologyDBMS ConceptsTech & HardwareLive DemoTeam

s sathwika

Log out

OverviewMethodologyDBMS ConceptsTech & HardwareLive DemoTeam

Methodology

Two phases: design the schema, then match inside the database

phase 1

Database design & schema

ER modeling

Donor, NGO, FoodListing, Volunteer and Pickup are modeled with clear relationships and cardinalities.

Normalization to 3NF

Localities, food types and NGO accepted types are split into their own tables, removing redundancy while keeping referential integrity.

Schema implementation

Primary/foreign keys, constraints and B-tree indexes are created by Flyway migrations on MySQL.

7. Query output of `match\_log` and `notifications`

